

CHAPITRE 4

Structures itératives

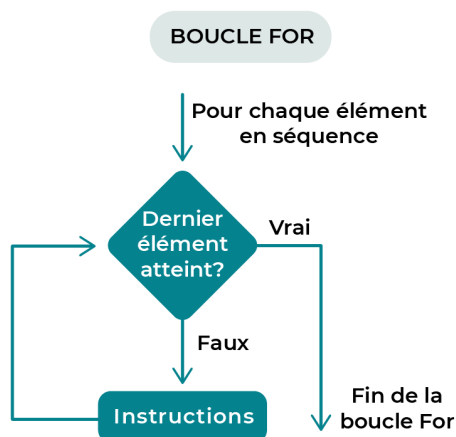
En programmation, il y a des ensembles d'instructions qu'il faut répéter plusieurs fois. Quand on a besoin de répéter un ensemble d'instructions, parfois on sait combien de fois on doit le répéter, d'autres fois on ne le sait pas à priori, on sait juste qu'il faut répéter le code jusqu'à ce qu'une certaine condition soit remplie. Pour tous ces usages, on va utiliser des boucles.

Les structures de répétition se classent en deux catégories : les *répétitions inconditionnelles* pour lesquelles le bloc d'instructions est à répéter un nombre donné de fois et les *répétitions conditionnelles* pour lesquelles le bloc d'instructions est à répéter autant de fois qu'une condition est vérifiée.

Pour comprendre l'exécution d'un code pas à pas on pourra utiliser : [Visualize code and get live help](http://pythontutor.com/visualize.html)
<http://pythontutor.com/visualize.html>

4.1. Répétition `for` : boucle inconditionnelle (parcourir)

Lorsque l'on souhaite répéter un bloc d'instructions un nombre déterminé de fois, on peut utiliser un *compteur actif*, c'est-à-dire une variable qui compte le nombre de répétitions et conditionne la sortie de la boucle.



C'est la structure introduite par le mot-clé `for` qui a la forme générale suivante (attention à l'**indentation** et aux **deux points**) :

```
for target in sequence:
    → instruction_1
    → instruction_2
    → ...
```

où `target` est le *compteur actif* et `sequence` est un itérateur (souvent généré par la fonction `range` ou une liste, un tuple, une chaîne de caractères, un dictionnaire). Tant que `target` appartient à `sequence`, on exécute les instructions `instruction_i`.

Quelques exemples où `sequence` est une liste, une chaîne de caractères ou l'itérateur `range` :

```
L = ["Qui", "Quo", "Qua"]
for truc in L:
    → print(truc)
```

Qui
Quo
Qua

```
L = "Minnie"
for c in L:
    → print(c)
```

M
i
n
n
i
e

```
for n in range(5):
    → print(n)
```

0
1
2
3
4

Il est possible d'imbriquer des boucles, c'est-à-dire que dans le bloc d'une boucle, on utilise une nouvelle boucle.

```
for x in [10,20,30,40]:
    for y in [3,7]:
        print(x+y,end=" ")
```

Dans ce petit programme x vaut d'abord 10, y prend la valeur 3 puis la valeur 7 (le programme affiche donc d'abord 13, puis 17). Ensuite $x = 20$ et y vaut de nouveau 3 puis 7 (le programme affiche donc ensuite 23, puis 27). Au final le programme affiche :

13, 17, 23, 27, 33, 37, 43, 47,

On peut utiliser une boucle pour ajouter des éléments à une liste avec la méthode `append`. Voici un exemple :

Création de la liste

```
L = [] # liste vide
for n in range(4):
    L.append(1/2**n)
print(L)
```

$\left[1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}\right]$ [1.0, 0.5, 0.25, 0.125]

enumerate Pour accéder en même temps à la position et au contenu on utilisera `enumerate(liste)`.

```
L = ["Riri", "Fifi", "Loulou"]
for idx,item in enumerate(L):
    print(item, idx)
```

Riri 0
Fifi 1
Loulou 2

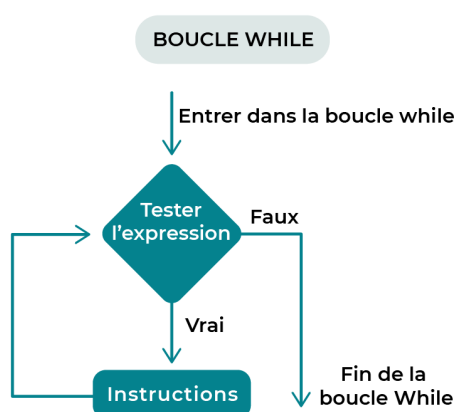
On peut modifier la valeur de départ de `enumerate` :

```
L = ["Riri", "Fifi", "Loulou"]
for idx,item in enumerate(L,start=101):
    print(item, idx)
```

Riri 101
Fifi 102
Loulou 103

4.2. Boucle `while` : répétition conditionnelle

La boucle `for` permet d'exécuter du code un nombre spécifique de fois, alors que la boucle `while` continue de s'exécuter jusqu'à ce qu'une certaine condition soit remplie.



`While` est la traduction de “tant que...”. Concrètement, la boucle s'exécutera tant qu'une condition est remplie (donc tant qu'elle renverra la valeur `True`).

Le constructeur `while` a la forme générale suivante (attention à l'**indentation** et aux **deux points**) :

```
while condition:
    instruction_1
    instruction_2
    ...
```

où `condition` représente des ensembles d'instructions dont la valeur est `True` ou `False`. Tant que la condition `condition` a la valeur `True`, on exécute les instructions `instruction_i`.

ATTENTION

Si la condition ne devient jamais fausse, le bloc d'instructions est répété indéfiniment et le programme ne se termine pas.

Affichage d'un compte à rebours. Tant que la condition $n > 0$ est vraie, on diminue n de 1. La dernière valeur affichée est $n = 1$ car ensuite $n = 0$ et la condition $n > 0$ devient fausse donc la boucle s'arrête.	<pre>n = 3 while n>0: → print(n) → n -= 1 print("Go!")</pre>	Output : <pre>3 2 1 Go!</pre>
Création de la liste $\left[1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}\right]$	<pre>n = 0 L = [] while n<4: → L.append(1/2**n) → n += 1 print(L)</pre>	<pre>[1.0, 0.5, 0.25, 0.125]</pre>
On divise n par 2 tant qu'il est pair (cela revient à supprimer tous les facteurs 2 de l'entier n). Par exemple, si $n = 168 = 2^3 \times 3 \times 7$, le programme affichera 21.	<pre>n = 168 while n%2==0: → n //= 2 print(n)</pre>	<pre>21</pre>
On calcule la somme des n premiers entiers (et on vérifie qu'on a bien $n(n+1)/2$).	<pre>n = 100 s,i = 0,0 while i<n: → i += 1 → s += i print(s) print(f'{n*(n+1)/2=}'))</pre>	<pre>5050 n*(n+1)/2=5050.0</pre>
Exemple classique de ce qu'on appelle une recherche de seuil : on recherche à partir de quelle valeur de n la somme $1 + 2 + 3 + \dots + n$ dépasse un million. Pour vérifier notre code, on peut afficher n, somme et aussi $n-1$ et somme-n.	<pre>n = 0 somme = 0 while somme < 10**6 : n += 1 somme += n print(f"{n=}, {somme=}") print(f"{n-1=}, {somme-n=}")</pre>	<pre>n=1414, somme=1000405 n-1=1413, somme-n=998991</pre>

4.3. ★ Ruptures de séquences

- Interrompre une boucle : `break`
Sort immédiatement de la boucle `for` ou `while` en cours contenant l'instruction :

```
for i in [1,2,3,4,5]:
    if i ==4:
        break
    print(i, end=" ")
print("Boucle interrompue pour i =", i)
```

```
1 2 3 Boucle interrompue pour i = 4
```

L'instruction `break` sort de la plus petite boucle `for` ou `while` englobante.

```
for lettre in 'AbcDefGhj':
    if lettre=='D':
        break
    print(lettre, end=" ")
```

A b c

- Court-circuiter une boucle : `continue`

Passe immédiatement à l'itération suivante de la boucle `for` ou `while` en cours contenant l'instruction; reprend à la ligne de l'en-tête de la boucle :

```
for i in [1,2,3,4,5]:
    if i ==4:
        continue
    print(i, end=" ")
# la boucle a sauté la valeur 4

1 2 3 5
```

L'instruction `continue` continue sur la prochaine itération de la boucle. Par exemple, supposons que nous voulions afficher $1/n$ pour n dans une liste. Si n vaut 0, le calcul $1/n$ sera impossible, il faudra donc sauter cette étape et passer au nombre suivant :

```
liste = [ -4, 2, 6, 0, 1, 3, 0, 10]
for n in liste:
    if n ==0:
        continue
    print(1/n, end=" ")

-0.25 0.5 0.16666666666666666 1.0
0.3333333333333333 0.1
```

Autre exemple :

1 3 5 Boucle terminée

```
a=0
while a<=5:
    a+=1
    if a%2==0:
        continue
    print(a, end=" ")
print("Boucle terminée")
```

Comparons les trois codes suivants :

<pre>for i in [1,2,3,4,5]: if i ==4: print("J'ai trouvé") print(i, end=" ") 1 2 3 J'ai trouvé 4 5</pre>	<pre>for i in [1,2,3,4,5]: if i ==4: print("J'ai trouvé") break print(i, end=" ") 1 2 3 J'ai trouvé</pre>	<pre>for i in [1,2,3,4,5]: if i ==4: print("J'ai trouvé") continue print(i, end=" ") 1 2 3 J'ai trouvé 5</pre>
--	--	---

- Utilisation avancée des boucles : la syntaxe complète des boucles autorise des utilisations plus rares. Les boucles `while` et `for` peuvent posséder une clause `else` qui ne s'exécute que si la boucle se termine normalement, c'est-à-dire sans interruption.

Les instructions de boucle ont une clause `else` qui est exécutée lorsque la boucle se termine par épuisement de la liste (avec `for`) ou quand la condition devient fausse (avec `while`), mais pas quand la boucle est interrompue par une instruction `break` :

```
for item in items:
    if something(item):
        do_st(item)
        break
else: # else associé à for et non au if !!!
    print("Not found") # exécuté si la boucle se termine mais non exécuté si termine par
    break

o while - else

y = 10 # un entier positif
x = y // 2
```

```

while x > 1:
    → if y % x == 0:
    → → print(y, "a pour facteur", x)
    → → break # voici l'interruption !
    → x -= 1
else :
    → print(y, "est premier.")
10 a pour facteur 5

```

o for - else

Un exemple avec le parcours d'une liste :

```

L = [2, 5, 9, 7, 11]
cible = 5
for i in L :
    → if i == cible :
    → → sauve = i
    → → break # voici l'interruption !
else : # else pour le for et non pas le if !
    → print(cible, "n'est pas dans", L)
    → sauve = None
# sauve vaut cible ou None
print("On obtient sauve =", sauve)
On obtient sauve = 5

```

Ceci est expliqué dans la boucle suivante, qui recherche des nombres premiers :

	2 est un nombre premier
for n in range(2,10):	3 est un nombre premier
→ for x in range(2,int(n/2)+1):	4 est égale à 2*2
→ → if n%x==0:	5 est un nombre premier
→ → → print(f"{n} est égale à {x}*{int(n/x)}")	6 est égale à 2*3
→ → → break	7 est un nombre premier
→ else:	8 est égale à 2*4
→ → print(f'{n} est un nombre premier')	9 est égale à 3*3